Practical – 10

Practical Title: Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Problem Definition: Develop secure login and logout functionality using PHP sessions. Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session timeout, and optional secure remember-me token.

Theory :

1. Are sessions started and destroyed securely?

PHP sessions are used to maintain the user's login status between pages. After successful login, the session ID is regenerated using session_regenerate_id(true). During logout, session data is cleared and the session is destroyed. The session cookie is also removed. These steps help protect the user's account and prevent unauthorized access.

2. Is password_verify() used correctly?

The password_verify() function checks the password entered by the user against the password hash stored in the database. If both match, the user can continue the login process. If they do not match, an error message is displayed. Passwords should be stored using password_hash() during registration. This helps protect user passwords in the database.

3. Is unauthorized dashboard access blocked?

The require_role() function checks whether the user is logged in and has the required role. Students can access student pages, while admin pages are restricted to admin users. If a student tries to open an admin page directly, access is denied. This prevents users from accessing pages they are not authorized to use.

4. Is session timeout implemented?

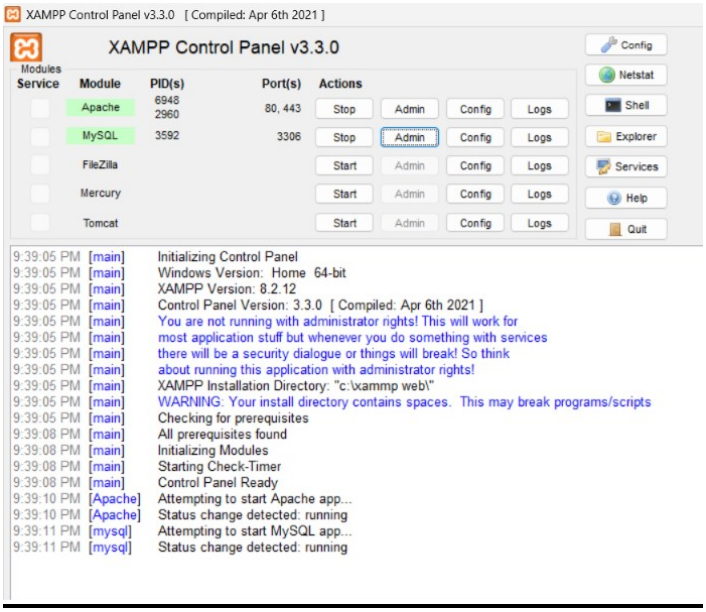
The system checks the user's role after verifying the password. If the role is student, the user is redirected to dashboard.php. If the role is admin, the user is redirected to admin_dashboard.php. Both users use the same login system with different roles. This makes authentication easier to manage and helps control access to different parts of the website.

5. Are admin and student users redirected to appropriate dashboards?

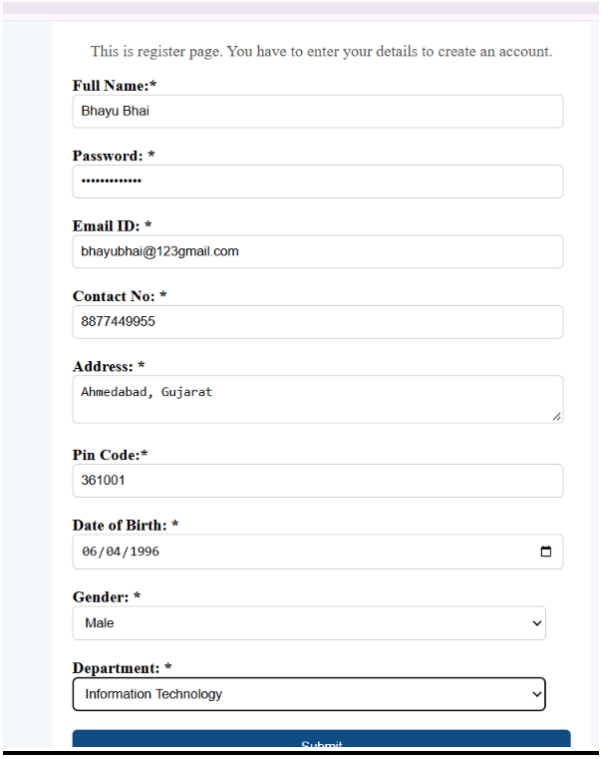
The system checks the user's role after verifying the password. If the role is student, the user is redirected to dashboard.php. If the role is admin, the user is redirected to

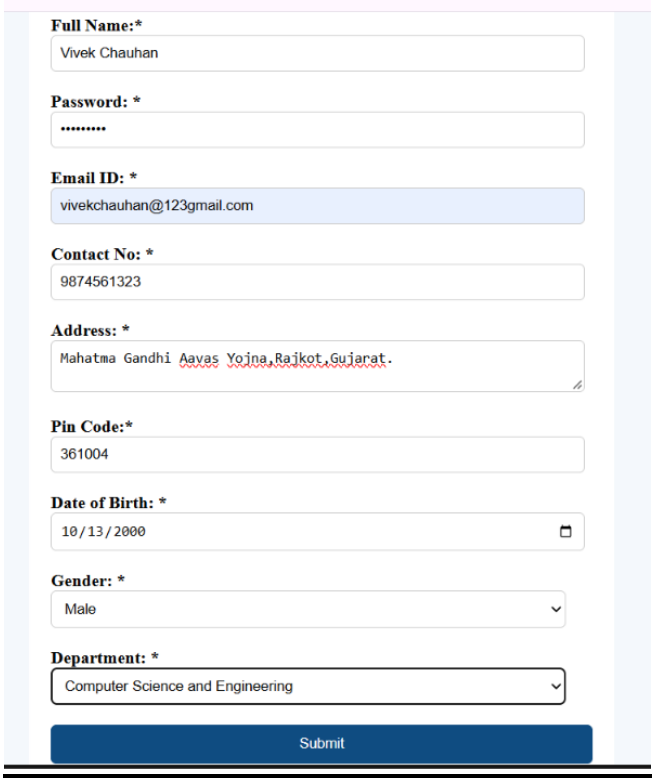
admin_dashboard.php. Both users use the same login system with different roles. This makes authentication easier to manage and helps control access to different parts of the website.

(1)Start XAMMP Local Host Server(APACHE and MYSQL Start):

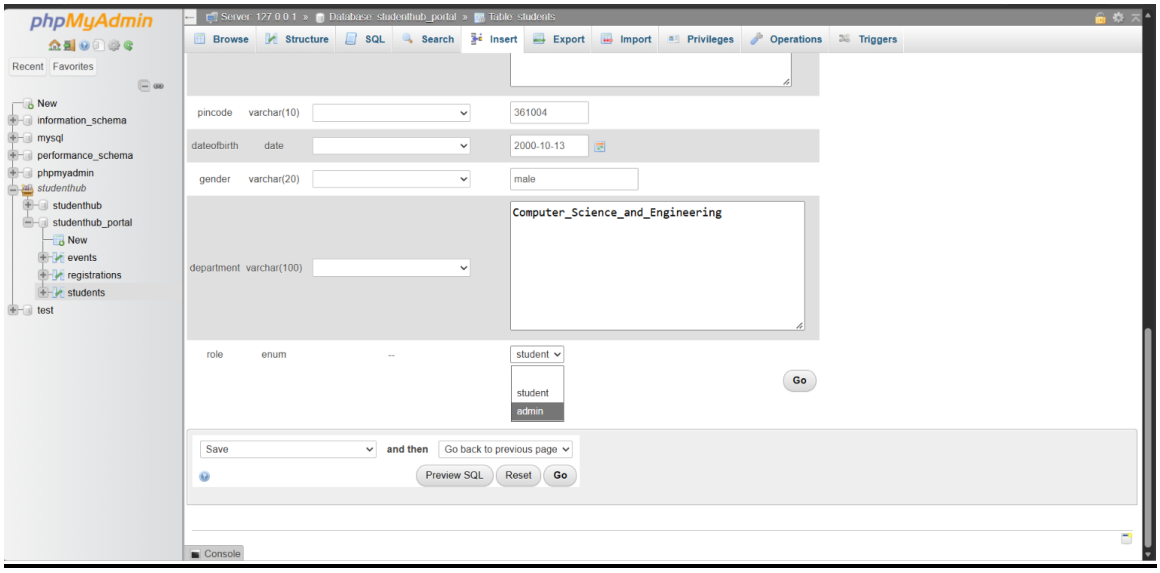


(2)Register with New Student:

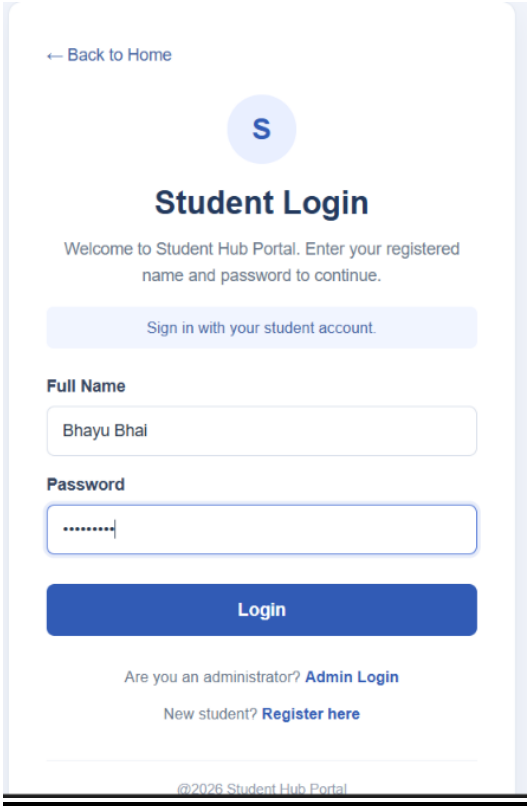




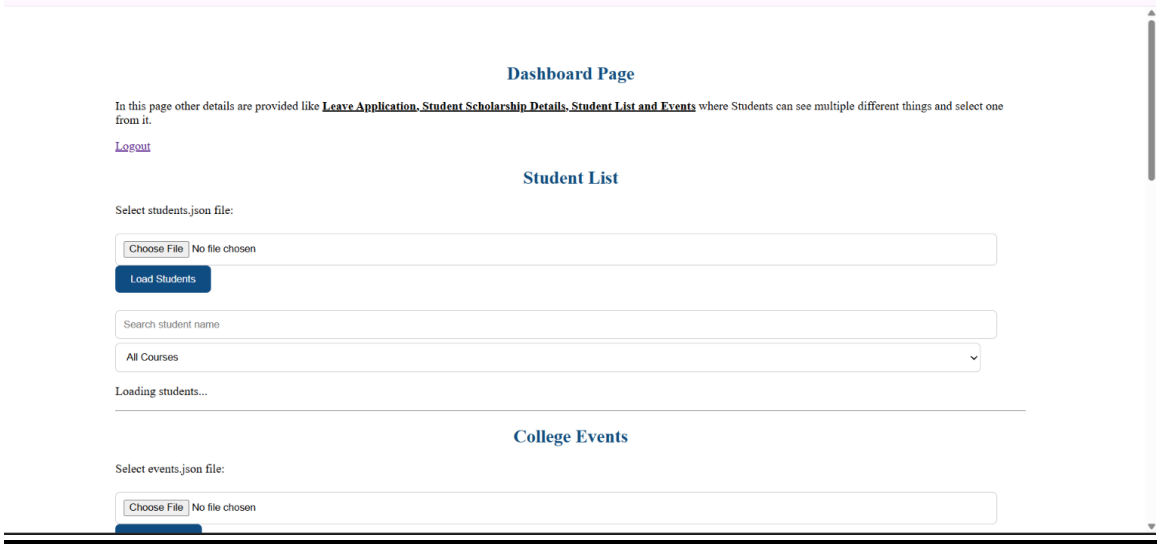
(3)Make Student role as Admin:



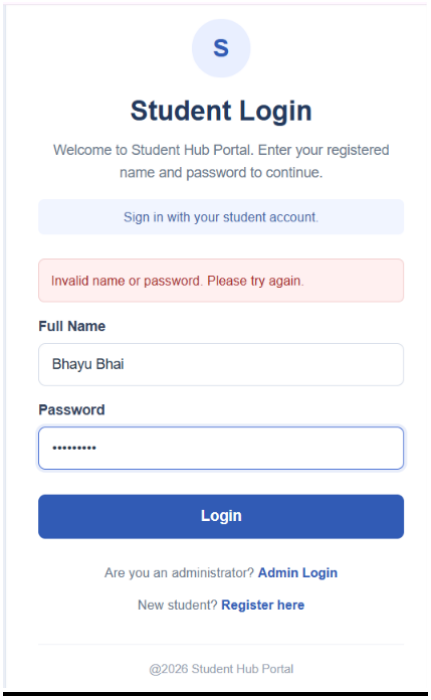
(4)Try to login as Student with Student username and password :



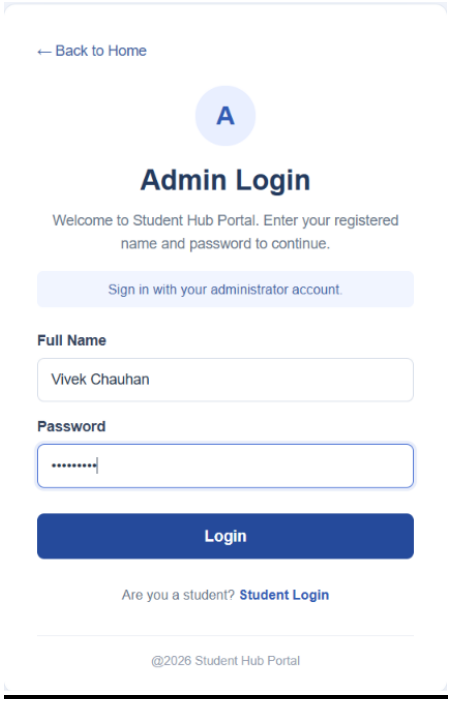
(5)Successful Login as Student:



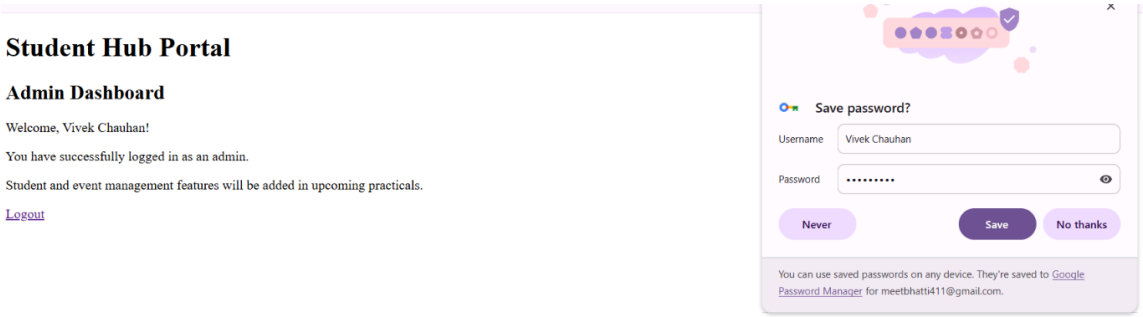
(6)Try to Login with wrong password:



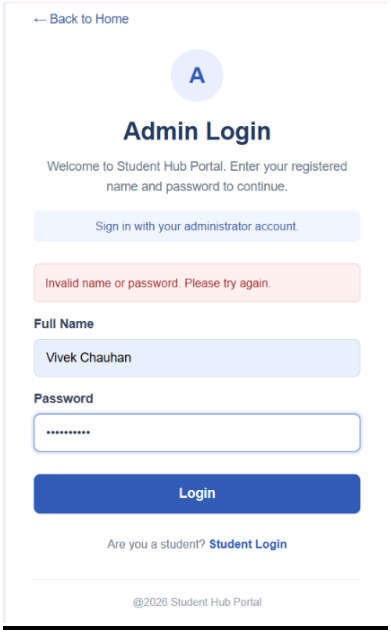
(7)Login as Admin:



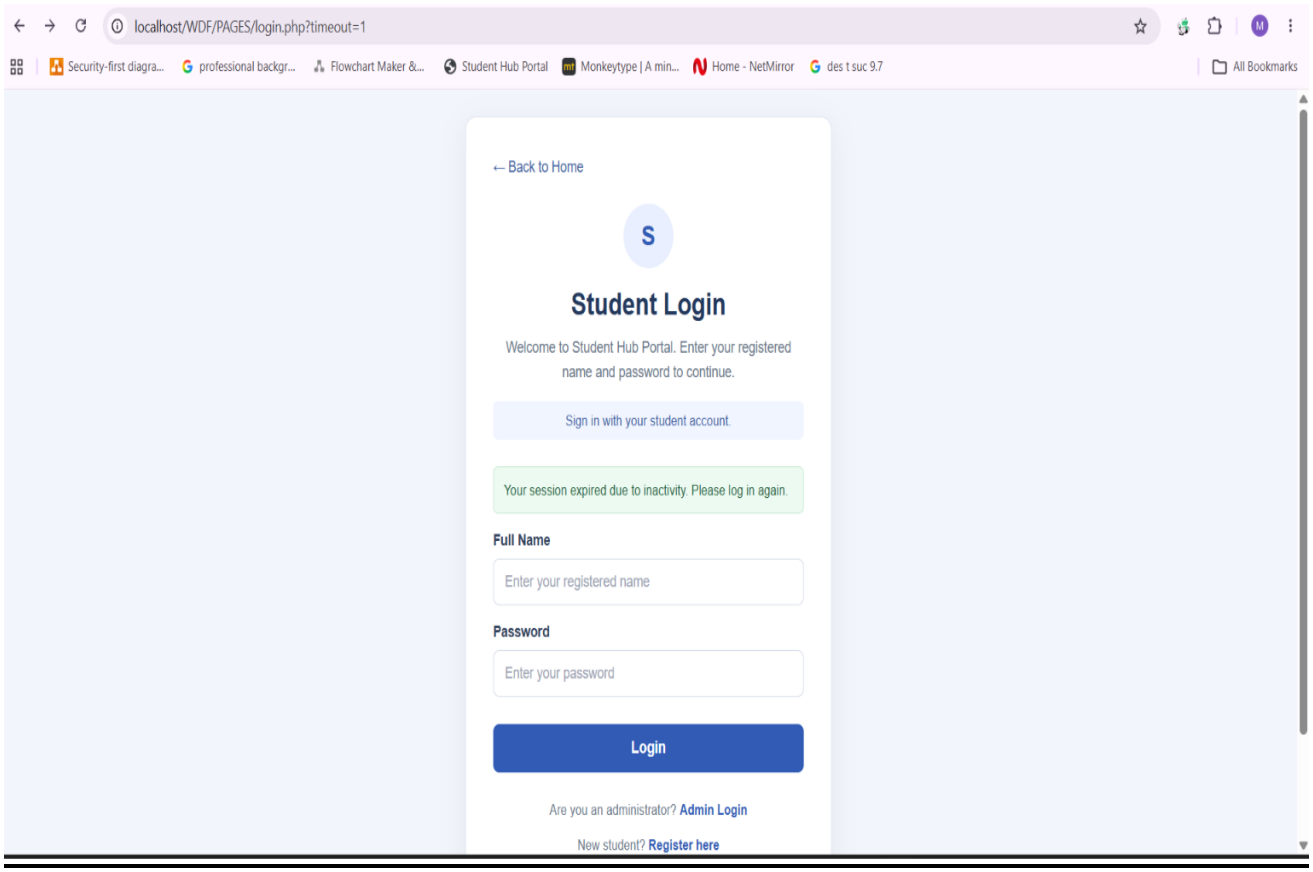
(6)Successful Login:



(7)Try with Wrong Password in Admin Login:



(8)Session Timeout shown in URL (login.php?timeout=1) :



GitHub Repository Link:

https://github.com/MeetBhatti/WDF.git

Learning Outcomes:

In this practical, I learned how to implement secure login and logout functionality using PHP and MySQL. I understood how PHP sessions maintain the login status of users across different pages. I learned to use password_verify() to check passwords securely and session_regenerate_id(true) to regenerate the session ID after login. I also understood role-based access control for separating student and admin permissions. I learned how session timeout and logout help protect accounts from unauthorized access. Finally, I practised testing successful login, incorrect passwords, dashboard access and session management in my Student Hub Portal.

Faculty Remarks: (Keep this blank for faculty)